

VORTEX TECH

Cyber Security Internship 2026

Week 3 Security Audit

Basic Security Audit of a Sample Website

Target: OWASP Juice Shop • URL: http://localhost:3000

Tool: OWASP ZAP 2.17.0 • Report date: 14 September 2026

0 HIGH	3 MEDIUM	1 LOW	1 INFO
-------------------	---------------------	------------------	-------------------

Professional presentation of the supplied audit report. Findings, evidence, risk levels and remediation content are preserved from the source report.

Program: Vortex Tech Cyber Security Internship 2026

Task: Week 3 of 4 — Intermediate

Assessment: Basic Security Audit of a Sample Website

Target: OWASP Juice Shop

Target URL: `http://localhost:3000`

Tool: OWASP ZAP 2.17.0

Report date: 14 September 2026

1. Executive Summary

A security audit was performed against a locally hosted OWASP Juice Shop instance. The application was assessed using OWASP ZAP, with the scan output retained as supporting evidence.

The ZAP report contains five alert types: two medium-risk CSP-related alerts, one medium-risk cross-domain/CORS alert, one low-risk Unix timestamp disclosure alert, and one informational alert identifying the application as a modern web application.

For the purpose of the assignment's requirement for 3+ findings, the two CSP alerts are treated as separate findings because they are separately reported by ZAP, while the category analysis makes clear that they belong to the same broader security-misconfiguration area. The audit therefore documents three practical vulnerability areas:

- Content Security Policy weaknesses
- Cross-domain/CORS misconfiguration
- Timestamp / information disclosure

The informational "Modern Web Application" alert is not treated as a vulnerability.

2. Methodology

The assessment followed the Week 3 task requirements:

- Use a legal practice target.
- Run OWASP Juice Shop locally.
- Use OWASP ZAP to assist the audit.
- Review the generated findings.
- Document each finding, discovery method, category, impact, and remediation.
- Preserve the ZAP report as evidence.

Testing workflow

```
Docker ↓ OWASP Juice Shop ↓ http://localhost:3000 ↓ OWASP ZAP scan ↓ Review alerts ↓ Classify findings ↓ Document impact + remediation
```

Finding 1 — CSP Directive Missing a Non-Fallback Directive

Severity: Medium

ZAP confidence: High

OWASP category: Security Misconfiguration — OWASP Top 10 2021 A05

CWE: CWE-693

Affected target: `http://localhost:3000/assets/public`

What was found

ZAP reported "CSP: Failure to Define Directive with No Fallback." The report identifies `frame-ancestors` and `form-action` as directives that do not fall back to `default-src`.

The observed CSP evidence was:

```
Content-Security-Policy: default-src 'none'
```

Steps taken to find it

- Started the local OWASP Juice Shop instance.
- Configured OWASP ZAP to assess `http://localhost:3000`.
- Allowed ZAP to crawl and passively inspect HTTP responses.
- Reviewed the Medium-risk alerts.
- Opened the CSP alert and inspected the request/response evidence.
- Confirmed that the response contained `default-src 'none'` but did not explicitly define the non-fallback directives identified by ZAP.

Potential real-world impact

An incomplete CSP can reduce the protection expected from the browser's Content Security Policy. In a real application, missing directives such as `frame-ancestors` or `form-action` can leave specific browser-controlled security behaviors less restricted than intended.

This can increase exposure to attacks such as clickjacking or unsafe form-submission scenarios when other application weaknesses are present.

Practical remediation

Define an explicit CSP policy containing the required non-fallback directives, rather than relying only on `default-src`.

For example, after testing application compatibility, a production policy should explicitly control the allowed framing and form destinations:

```
Content-Security-Policy: default-src 'self'; frame-ancestors 'self'; form-action 'self'
```

The exact policy must be adapted to the application's legitimate resources and functionality.

Finding 2 — Content Security Policy Header Not Set

Severity: Medium

ZAP confidence: High

OWASP category: Security Misconfiguration — OWASP Top 10 2021 A05

CWE: CWE-693

Affected endpoint: GET `http://localhost:3000/sitemap.xml`

What was found

ZAP reported "Content Security Policy (CSP) Header Not Set" for an application response.

The response headers shown in the report include headers such as `X-Content-Type-Options` and `X-Frame-Options`, but the reported endpoint did not provide a `Content-Security-Policy` header.

Steps taken to find it

- Ran the ZAP scan against the local Juice Shop application.
- Reviewed the passive scanner alerts.
- Selected the CSP Header Not Set finding.

- Examined the affected request:

```
GET http://localhost:3000/sitemap.xml
```

- Inspected the response headers.
- Confirmed that the endpoint did not contain a Content-Security-Policy response header.

Potential real-world impact

CSP provides an additional browser-side security layer that can help reduce the impact of certain XSS and data-injection attacks. If CSP is missing, an application loses this defense-in-depth control.

CSP is not a replacement for secure coding or output encoding, but its absence can make successful client-side injection attacks more damaging.

Practical remediation

Configure the web/application server to send a tested Content-Security-Policy header on applicable responses.

Example starting point:

```
Content-Security-Policy: default-src 'self'; object-src 'none'; frame-ancestors 'self'; form-action 'self'
```

The policy should be tested against the application's required JavaScript, CSS, image, font, API, and third-party resources before deployment.

Finding 3 — Cross-Domain / CORS Misconfiguration

Severity: Medium

ZAP confidence: Medium

OWASP mapping reported by ZAP: OWASP Top 10 2021 A01 — Broken Access Control

CWE: CWE-264

Affected endpoint: GET http://localhost:3000/robots.txt

What was found

ZAP identified a Cross-Domain Misconfiguration caused by an overly permissive CORS response.

The key evidence was:

```
Access-Control-Allow-Origin: *
```

ZAP explains that this configuration can permit cross-domain read requests from arbitrary third-party origins for unauthenticated resources. ZAP also notes that browser protections reduce the risk for authenticated APIs.

Steps taken to find it

- Ran the ZAP scan against http://localhost:3000.
- Reviewed the Medium-risk findings.
- Selected the Cross-Domain Misconfiguration alert.
- Examined the affected request:

```
GET http://localhost:3000/robots.txt
```

- Inspected the response headers.
- Confirmed the wildcard CORS policy:

```
Access-Control-Allow-Origin: *
```

Potential real-world impact

An overly broad CORS policy can allow a third-party origin to read responses that the application makes available cross-origin.

For applications containing sensitive unauthenticated or weakly protected endpoints, this can increase the risk of unauthorized data access. The impact is especially important when cross-origin access is not actually required by the application's design.

Practical remediation

Replace the wildcard CORS policy with an allow-list containing only trusted origins, or remove CORS headers where cross-origin access is unnecessary.

For example:

```
Access-Control-Allow-Origin: https://trusted.example
```

Do not use * for sensitive cross-origin resources.

Finding 4 — Unix Timestamp Disclosure

Severity: Low

ZAP confidence: Low

Category: Information Disclosure / Sensitive Data Exposure

CWE: CWE-497

Affected endpoint: GET http://localhost:3000/styles.css

What was found

ZAP reported "Timestamp Disclosure - Unix" and identified the value:

```
1528301887
```

ZAP evaluated this value as:

```
2018-06-06 21:18:07
```

Steps taken to find it

- Ran the ZAP scan against the local Juice Shop target.
- Reviewed the Low-risk findings.
- Opened the Timestamp Disclosure alert.
- Examined the affected request:

```
GET http://localhost:3000/styles.css
```

- Inspected the response content and located the Unix timestamp.
- Confirmed that the timestamp could be converted to a human-readable date.

Potential real-world impact

A single timestamp is normally low impact. However, timestamps can reveal development, deployment, build, or modification information. If attackers can collect many such values, they may infer application timelines or correlate information with other public data.

The ZAP report therefore recommends manually confirming whether the timestamp is sensitive and whether timestamps can be aggregated into exploitable patterns.

Practical remediation

Remove unnecessary timestamps from publicly served resources or ensure that build/deployment metadata is not exposed unless it is required.

If timestamps are necessary, review whether the information can reveal sensitive development or infrastructure details.

Modern Web Application

ZAP also identified "Modern Web Application" as an informational alert.

This is not treated as one of the security findings because ZAP explicitly classifies it as informational and states that no changes are required.

It is therefore excluded from the vulnerability count.

Finding	ZAP Risk	Confidence	Category
CSP directive with no fallback	Medium	High	Security Misconfiguration
CSP header not set	Medium	High	Security Misconfiguration
Cross-domain/CORS misconfiguration	Medium	Medium	CORS / Access-Control Misconfiguration
Unix timestamp disclosure	Low	Low	Information Disclosure
Modern Web Application	Informational	Medium	Not a vulnerability

Total ZAP alert types: 5

Medium-risk alert types: 3

Low-risk alert types: 1

Informational alert types: 1

High-risk findings: 0

This assessment is based primarily on the provided OWASP ZAP scan output. Automated scanners do not prove that an application is free of vulnerabilities.

The ZAP report does not demonstrate a successful XSS exploit, broken-login test, or sensitive-data discovery through browser DevTools. Therefore, those should not be claimed as tested findings unless separate evidence/screenshots are added.

The task brief also suggests manual exploration of search/forms, authentication behavior, and API responses. If those activities were actually performed, their screenshots or notes should be added to the repository.

The original ZAP HTML report is stored at:

`evidence/zap-report.html`

Important evidence visible in the report includes:

- Target: `http://localhost:3000`
- ZAP version: 2.17.0
- CSP-related findings
- Access-Control-Allow-Origin: *
- Unix timestamp 1528301887
- Risk/confidence levels and affected endpoints

- [] Public GitHub repository created.
- [] Repository has a clear name such as `vortextech-cybersec-week3`.
- [] `README.md` included.

- [] Structured audit report included.
- [] ZAP HTML evidence included.
- [] Screenshots added if available/required.
- [] Docker/OWASP Juice Shop run instructions included.
- [] Week 3 submission form completed with repository URL.

END OF REPORT